

《数据分析与数据挖掘》课程项目报告

基于 MovieLens 100K 数据集的 智能电影推荐系统设计与实现

——FilmTrace 推荐系统

小组成员：吕璇 陈琳 傅梦娇

指导教师：【教师姓名】

提交日期：2026 年 6 月

原创性声明

本小组郑重声明：所提交的课程项目报告《基于 MovieLens 100K 数据集的智能电影推荐系统设计》及配套源代码均为本小组成员独立完成的原创性成果。报告中所使用的 MovieLens 100K 数据集为 GroupLens 研究组公开发布的标准数据集；所引用或借鉴的他人研究成果、开源代码与第三方库（如 scikit-learn、pandas、Streamlit 等）均已在正文或参考文献中明确标注出处。除文中已注明引用的内容外，本报告不包含任何未经标注的他人作品，亦不存在抄袭、剽窃或代写行为。

本小组已知悉课程关于学术诚信与查重的相关规定（实验报告查重率不得超过 20%），并对本声明的真实性承担相应责任。

小组成员签名（手写）：

吕璇：_____ 陈琳：_____ 傅梦娇：_____

日期：_____年_____月_____日

目录

目录.....	3
1 引言.....	6
1.1 研究背景与意义.....	6
1.2 问题定义.....	6
1.3 项目目标与功能范围.....	6
2 相关工作.....	7
3 数据集与数据预处理.....	8
3.1 数据来源与规模.....	8
3.2 数据字段说明.....	8
3.3 数据清洗流程.....	8
3.4 训练/测试集划分.....	9
3.5 探索性数据分析.....	9
4 系统需求分析.....	11
5 系统设计.....	12
5.1 总体架构设计.....	12
5.2 数据流说明.....	12
5.3 数据库与逻辑数据模型设计.....	12
6 推荐算法设计.....	15
7 系统实现.....	17
7.1 系统欢迎页与角色入口.....	17
7.2 用户注册、登录与偏好选择.....	17
7.3 用户端首页、电影库与电影详情.....	18
7.4 个性化推荐与相似电影推荐.....	21
7.5 用户评分、评价与收藏管理.....	24
7.6 用户画像与数据可视化.....	26
7.7 管理员后台功能实现.....	28
8 实验设计与模型评估.....	35
8.1 评估协议与指标.....	35
8.2 实验设置与参数.....	35
8.3 模型评估结果.....	35
8.4 相关性阈值敏感性分析.....	36
8.5 超参数敏感性分析.....	37
8.6 NeuralCF 训练过程可视化.....	37
9 结果分析.....	39
9.1 定量结果分析.....	39
9.2 错误案例与原因分析.....	39
9.3 覆盖率与多样性分析.....	39
9.4 冷启动与定性结果.....	40
9.5 局限性小结.....	40
10 项目测试与运行说明.....	41
10.1 运行环境与依赖.....	41
10.2 数据与目录结构.....	41
10.3 运行步骤.....	41
10.4 可复现性声明.....	41
11 团队分工.....	42
12 Git 版本控制与项目管理.....	43
13 结论与展望.....	44
13.1 主要工作与发现.....	44
13.2 系统价值与局限性.....	44

13.3 未来工作..... 44

参考文献..... 45

附录..... 46

附录 A 实验记录与补充图表..... 46

附录 B 实验记录表（含未达预期的尝试）..... 46

附录 C 可复现性与随机种子..... 46

摘要

随着在线影视内容规模的迅速增长，用户面临严重的信息过载问题，个性化电影推荐成为提升内容分发效率的关键技术。本项目以 GroupLens 发布的 MovieLens 100K 数据集（943 名用户、1,682 部电影、100,000 条 1-5 分评分，矩阵稀疏度约 93.7%）为基础，设计并实现了一个名为 FilmTrace 的智能电影推荐系统。在算法层面，项目实现并系统对比了 9 类传统模型，涵盖全局/用户/物品均值、正则化偏置、随机与最受欢迎等 6 类基线，以及基于用户的协同过滤（UserCF）、基于物品的协同过滤（ItemCF）、基于偏置残差的 TruncatedSVD 矩阵分解和元数据混合模型；在此基础上，进一步实现了基于 PyTorch 的神经协同过滤（NeuralCF）深度学习模型，采用 MSE 损失与 AdamW 优化器进行端到端训练，并从损失函数、优化器与训练过程可视化等角度与传统方法对比。评估采用按用户时间序列划分的训练/测试集（random_state=42），从评分预测与 Top-N 排序两个角度，使用 RMSE、MAE 以及 Precision@10、Recall@10、HitRate@10、NDCG@10 和目录覆盖率等指标进行衡量。实验表明，偏置残差 SVD 取得了最优的 RMSE（0.9826）与 MAE（0.7763），略优于深度模型 NeuralCF（RMSE 1.0080、MAE 0.8003）；而在全目录离线 Top-N 协议下，流行度类基线在排序指标上反而领先于个性化模型，本报告对这一反直觉现象给出了基于评估协议与数据稀疏性的分析。在系统层面，项目基于 Streamlit 与 SQLite 实现了包含用户端与管理端完整推荐系统，集成账号体系、偏好选择、个性化推荐、评分收藏、用户画像、数据可视化与后台管理等功能，具有较强的工程完整性与实用价值。

关键词：电影推荐系统；MovieLens 100K；协同过滤；矩阵分解；Streamlit；数据可视化

1 引言

1.1 研究背景与意义

近年来，在线影视平台的内容规模呈爆发式增长，用户在海量影片中检索符合自身口味的内容变得愈发困难，这种“信息过载”现象显著降低了用户的内容获取效率与平台的内容分发效率。推荐系统通过对用户历史行为建模，主动为用户筛选并排序其可能感兴趣的内容，已成为缓解信息过载、提升用户体验与平台留存的核心技术。

电影推荐是推荐系统最经典的应用场景之一：电影具有类型、年代等结构化属性，用户评分行为相对规律，且 MovieLens 系列数据集长期作为协同过滤与矩阵分解算法研究的标准基准，便于将实验结果与既有研究进行横向比较。因此，围绕 MovieLens 100K 构建一个完整的推荐系统，既具有算法研究价值，也具有工程实践意义。

1.2 问题定义

本项目需要解决的核心问题可归纳为两类：

- 1) 评分预测问题：给定用户的历史评分记录，预测其对未评分电影的评分，本质上是一个在稀疏评分矩阵上的回归问题，以 RMSE、MAE 衡量预测误差；
- 2) Top-N 排序推荐问题：在用户尚未评分的电影集合中，筛选并排序出最值得推荐的 N 部电影，以 Precision@K、Recall@K、HitRate@K、NDCG@K 等排序指标衡量推荐列表质量。

与之对应的实际痛点包括：新注册用户缺乏历史评分，难以直接使用协同过滤（冷启动问题）；普通用户难以了解平台整体评分分布与自身观影偏好；运营方需要对电影库、用户数据与推荐算法参数进行集中管理与评估。

1.3 项目目标与功能范围

围绕上述问题，本项目设定如下目标：

- 算法目标：在 MovieLens 100K 上实现并统一对比 6 类基线、UserCF、ItemCF、SVD 与混合模型共 9 类传统推荐算法，并引入基于 PyTorch 的 NeuralCF 深度学习模型，分析传统方法与深度方法在评分预测与 Top-N 排序上的表现差异；
- 评估目标：建立涵盖 RMSE/MAE 与 Precision@K/Recall@K/HitRate@K/NDCG@K/覆盖率的多维评估体系，并完成相关性阈值与关键超参数的敏感性分析；
- 系统目标：基于 Streamlit 与 SQLite 构建面向用户与管理员的完整推荐系统 FilmTrace，通过“偏好类型选择”缓解冷启动，通过“用户画像/数据可视化”提升可解释性，通过管理员后台实现电影库、用户、评分、算法与评估的统一管理；
- 工程目标：保证项目的可复现性（固定随机种子、提供依赖环境文件与运行说明、使用 Git 进行全周期版本管理）。

2 相关工作

协同过滤 (Collaborative Filtering, CF) 是推荐系统中应用最广泛的一类方法, 其基本假设是“兴趣相近的用户会喜欢相似的物品”。基于邻域的协同过滤可分为基于用户 (User-Based CF) 与基于物品 (Item-Based CF) 两类。Sarwar 等人系统地分析了基于物品的协同过滤在大规模数据集上的可扩展性与推荐质量, 指出物品相似度通常比用户相似度更稳定, 更适合工业级推荐场景[2]。Herlocker 等人则较早对协同过滤推荐系统的评估方法进行了系统性总结, 强调单一的预测精度指标不足以反映推荐列表的排序质量与用户体验[4]。

随着 Netflix Prize 竞赛的推动, 矩阵分解 (Matrix Factorization) 成为评分预测任务的主流方法。Koren 等人系统总结了引入用户/物品偏置项的隐因子模型, 指出“偏置项 + 隐因子”的组合能够显著提升评分预测精度[3]。本项目实现的 SVD 模型即借鉴了这一思路, 采用“先用偏置基线拟合整体水平、再对评分残差进行 TruncatedSVD 低秩分解”的两阶段建模策略, 以更好地适应稀疏评分矩阵。

在评估协议方面, Cremonesi 等人指出, 在离线 Top-N 评估中, 未被用户评分的物品并非真实负样本, 全目录排序评估容易使流行度基线在 Precision/Recall 等指标上表现优异[5]。这一结论与本项目在结果分析中观察到的“流行度基线在 Top-N 指标上领先于个性化模型”的现象高度一致, 是理解本项目实验结果的重要理论依据。

为缓解冷启动与数据稀疏问题, 混合推荐 (Hybrid Recommendation) 将协同过滤与基于内容/元数据的方法相结合。Burke 对混合推荐系统进行了系统的综述与实验分析, 归纳了加权、切换、特征组合等多种混合策略[6]。近年来, He 等人提出的神经协同过滤 (Neural Collaborative Filtering, NCF) 通过嵌入向量与多层感知机建模用户 - 物品之间的非线性交互, 进一步拓展了协同过滤的表达能力[7]。本项目在传统的基线、UserCF、ItemCF、SVD 与混合模型之外, 进一步实现了基于 PyTorch 的 NeuralCF 深度学习模型, 并将其纳入统一的评估与对比, 以考察深度方法在该数据集上相对传统方法的实际表现。MovieLens 系列数据集由 GroupLens 研究组持续维护, 是推荐系统研究中引用最广泛的公开基准之一[1]。

3 数据集与数据预处理

3.1 数据来源与规模

本项目使用 GroupLens 研究组发布的 MovieLens 100K 数据集。该数据集包含 943 名用户对 1,682 部电影的 100,000 条评分记录，评分为 1 - 5 的整数等级，并附带评分发生的 Unix 时间戳。原始数据主要由三个文件构成：u.data（评分记录：用户、电影、评分、时间戳）、u.item（电影元数据：片名、上映日期、IMDb 链接及 19 个类型的二值标记）、u.user（用户人口统计信息，主要用于探索性分析）。表 2 给出了数据集的基本统计信息。

表 2 MovieLens 100K 数据集基本统计

统计项	数值	说明
用户数	943	至少评分过 20 部电影
电影数	1,682	含 19 个类型标记
评分总数	100,000	评分取值 1 - 5 的整数
评分矩阵规模	943 × 1,682	理论可填充单元约 158.6 万
实际填充率	约 6.3%	稀疏度约 93.7%
评分范围	1 - 5	整数等级

3.2 数据字段说明

表 1 列出了本项目实际使用的关键字段及其来源与用途。

表 1 主要数据字段说明

字段	来源	用途
user_id / movie_id	u.data / u.item	用户与电影标识，用于构建用户 - 电影评分矩阵
rating	u.data	1 - 5 评分，作为模型训练与评估的标签
timestamp	u.data	评分时间戳，用于按用户时间序列划分训练/测试集
title / release_year	u.item	电影片名与上映年份，用于电影库展示与年代分析
genres (19 类)	u.item	类型二值标记，用于类型分布统计与混合模型特征
review	系统新增 (ratings 表)	用户撰写的文字评价，写入 SQLite

3.3 数据清洗流程

数据预处理由 src/data_cleaning.py 与 src/preprocessing.py 实现，主要步骤如表 3 所示。

表 3 数据预处理流程

步骤	处理内容
字段类型转换	将 user_id、movie_id、rating、timestamp 统一转换为数值类型，非法值置为缺失并剔除
缺失值处理	校验评分必备字段 (user_id/movie_id/rating/timestamp) 的完整性，剔除关键字段缺失的记录

步骤	处理内容
重复值处理	以 (user_id, movie_id) 为主键，确保同一用户对同一电影仅保留一条评分记录
片名年份解析	使用正则表达式从片名中提取上映年份（“(YYYY)”），生成 release_year 字段
类型字段解析	将 u.item 中 19 列类型二值标记解析为 genres 类型列表，供展示与建模使用
评分范围核验	校验评分严格落在 1 - 5 区间，超出范围的记录视为异常并处理
海报资源匹配	src/posters.py 对片名规范化后匹配本地“电影照片/”目录中的海报，未匹配时使用统一占位图

需要说明的是，由于 MovieLens 100K 本身是经过 GroupLens 清洗的高质量数据集，本项目的清洗工作以类型转换、字段解析与一致性校验为主；推荐系统场景一般不涉及图像/文本类的数据增强（data augmentation），因此本项目未采用数据增强策略，而是通过“偏置正则化”与“偏置残差分解”等建模手段应对数据稀疏问题。

3.4 训练/测试集划分

主实验采用按用户时间序列划分（per-user temporal split）：对每位用户的评分按 timestamp 升序排列，将较早的评分作为训练集、较晚的评分作为测试集（测试比例 TEST_SIZE=0.2）。该方式避免了随机划分可能引入的“用未来数据预测过去”的信息泄漏，同时保证每位被评估用户都具有训练历史。作为参考，项目还实现了全局时间划分与随机划分（src/preprocessing.py 中的 split_ratings / time_based_split）。所有随机过程统一固定随机种子 RANDOM_STATE=42，以保证结果可复现。

3.5 探索性数据分析

在建模前，项目对数据进行了探索性分析。图 1 给出了评分分布直方图，图 2 给出了各电影类型的数量分布。

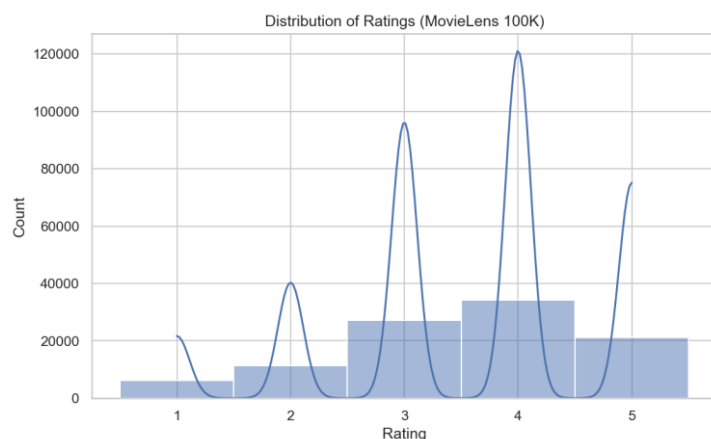


图 1 MovieLens 100K 评分分布直方图

图 1 显示评分主要集中在 3 - 4 分、整体呈左偏分布，说明用户倾向于对自己观看的电影给出中性偏正面的评价。这一分布特征意味着全局均值、用户均值等简单基线在 RMSE/MAE 上也能取得不差的成绩，是后文讨论个性化模型与基线差距时的重要背景。

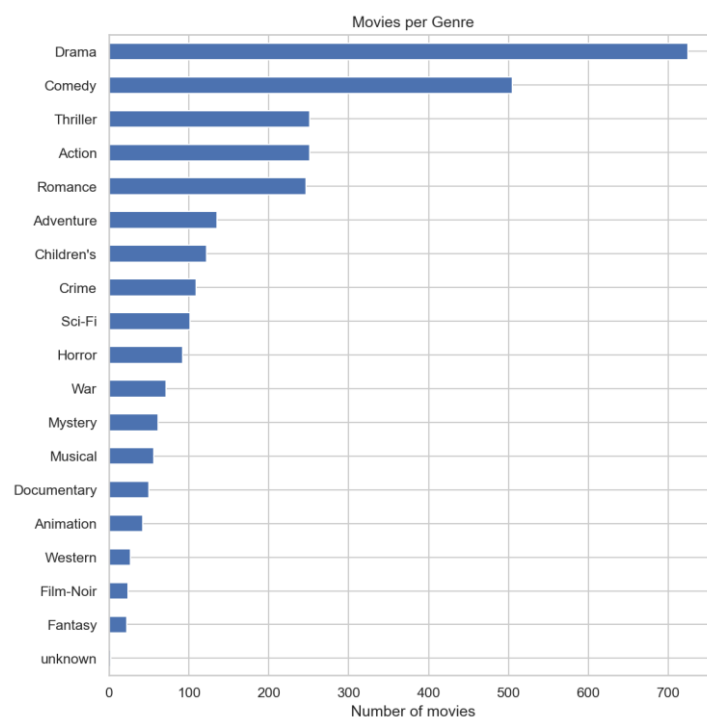


图 2 电影类型数量分布

图 2 显示 Drama、Comedy、Action 等类型的电影数量明显领先，类型分布存在显著的不均衡，这种长尾特征也是后文“个性化与目录覆盖率之间存在权衡”现象的数据背景之一。

4 系统需求分析

在算法研究之外，本项目还需将推荐能力落地为可运行的 Web 系统。为此，先从用户端、管理员端、数据管理、算法评估、可视化与可复现性等方面梳理系统功能需求，如表 4 所示。

表 4 功能模块需求表

需求类别	功能模块	具体要求
用户端	账号与偏好	支持用户注册、登录，并在注册时选择偏好电影类型以缓解冷启动
用户端	浏览与推荐	提供电影库浏览、电影详情、热门电影、个性化推荐与相似电影推荐
用户端	交互与画像	支持评分、文字评价、收藏管理，并基于行为生成用户画像与数据可视化
管理员端	数据管理	支持电影库、用户、评分记录的查询与管理（CRUD）
管理员端	算法与评估	支持推荐算法配置、模型评估（MAE/RMSE 等）与系统统计分析
数据管理	持久化	对账号、收藏、偏好、画像、评价等数据进行持久化存储
可复现性	工程规范	固定随机种子、提供依赖环境文件与运行说明、使用 Git 进行版本管理

从需求类型看，用户端以“浏览—推荐—反馈”为主线，强调个性化与可解释性；管理员端以“管理—配置—评估”为主线，强调对数据与算法的可控性；数据与可复现性需求则保证系统状态可持久化、实验结果可重复。

5 系统设计

5.1 总体架构设计

FilmTrace 采用“单体 Streamlit 应用 + SQLite/本地文件存储”的架构，不引入独立的后端 API 服务或在线深度学习推理服务，以契合课程项目的规模与可复现要求。系统总体架构如图 3 所示，可自上而下划分为六个层次。

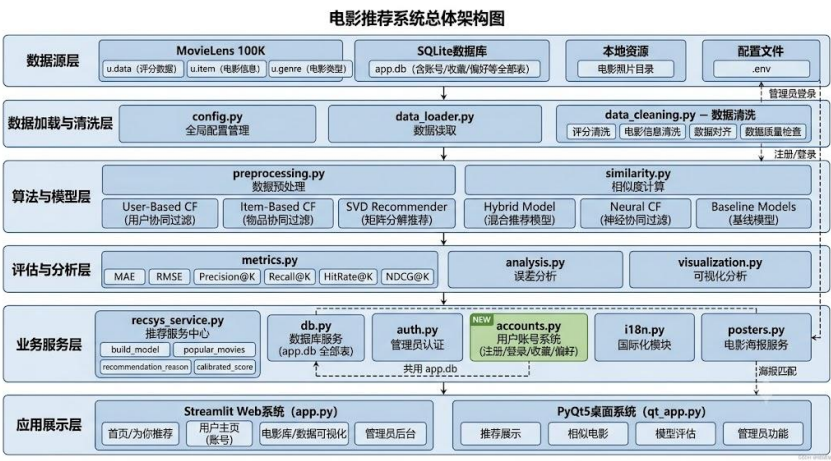


图 3 电影推荐系统总体架构图

各层职责如下：

- 表现层：基于 Streamlit 渲染用户端与管理端界面，负责页面导航、表单交互与图表展示；
- 功能模块层：组织用户端（账号认证、偏好选择、电影库、个性化推荐、评分收藏、用户画像、数据可视化）与管理端（电影库管理、用户管理、评分记录、算法配置、模型评估、系统统计）的业务逻辑；
- 数据处理层：完成 MovieLens 100K 的加载、清洗、特征预处理、评分矩阵构建、训练/测试划分与本地海报匹配（data_loader/data_cleaning/preprocessing/posters）；
- 推荐算法层：承载 9 类推荐模型（基线、UserCF、ItemCF、SVD、Hybrid）及在线推荐服务 recsys_service；
- 评估层：实现 RMSE/MAE 与 Precision@K/Recall@K/HitRate@K/NDCG@K/覆盖率等评估指标（metrics）及结果分析（analysis）；
- 数据存储与可视化层：以 SQLite (app.db) 持久化账号、收藏、偏好、画像、评价等数据，并通过 visualization 模块渲染统计图表。

5.2 数据流说明

系统的主数据流为：原始 MovieLens 文件经数据处理层加载清洗后形成评分矩阵与电影元数据；用户在表现层产生的注册、偏好、评分、收藏等行为经功能模块层写入 SQLite；推荐算法层结合评分矩阵、元数据与用户实时行为计算推荐结果，再经表现层以海报卡片形式呈现；评估层则在离线阶段对各模型进行统一评估，评估结果可在管理员端“模型评估”页面查看。

5.3 数据库与逻辑数据模型设计

系统采用 SQLite 作为物理数据库（app.db），相关读写逻辑统一封装在 src/db.py 中。数据库共包含 8 张表，其结构如表 5 所示；图 4 给出了主要实体及其关系的 ER 图。

表 5 SQLite 数据表结构概览

数据表	主键	主要字段与说明
movies	movie_id	title、data（电影元数据 JSON），存储电影库信息
users	user_id	n_ratings、avg_rating，MovieLens 用户的评分统计
ratings	(user_id, movie_id)	rating、timestamp、review，用户评分与文字评价
accounts	account_id	username、email、password_hash、movielens_user_id，系统登录账号
favorites	(account_id, movie_id)	created_at，用户收藏记录
user_preferences	account_id	genres、seed_movie_ids，注册时选择的偏好类型与种子电影
user_profiles	account_id	top_genres、year_range、preferred_min_rating，用户画像
admin_audit_log	id	ts、operation、administrator、target，管理员操作审计日志

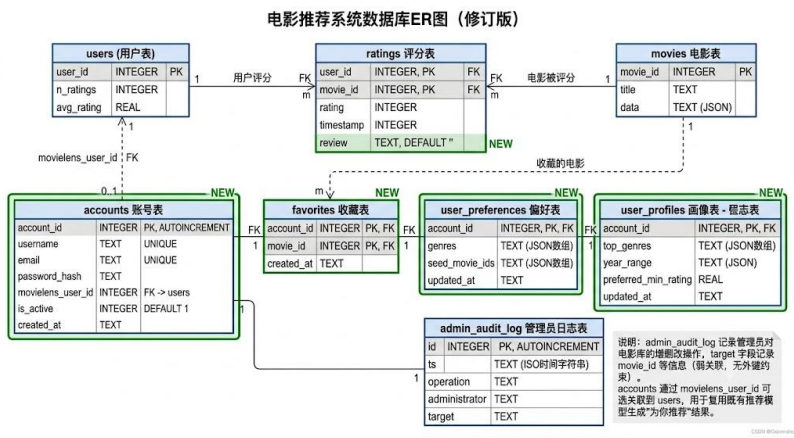


图 4 电影推荐系统数据库 ER 图

主要实体及关系说明如下：

- accounts 是系统登录账号实体，通过 movielens_user_id 与 MovieLens 的 users 实体建立关联，使系统账号能够复用 MovieLens 用户的历史评分；
- ratings 记录用户对电影的评分与评价，与 users（user_id）和 movies（movie_id）构成多对多关系的连接实体；
- favorites、user_preferences、user_profiles 均以 account_id 与 accounts 关联，分别承载收藏、偏好与画像数据，体现“一个账号对应一份偏好/画像、对应多条收藏”的关系；
- admin_audit_log 独立记录管理员的写操作，用于权限留痕与审计，不与具体业务实体强约束。

需要说明的是，movies、users、ratings 三张表在系统首次运行时由原始 MovieLens 数据落库

(src/db.py 的初始化与 seed 逻辑)，属于真实物理存储；accounts、favorites、user_preferences、user_profiles、admin_audit_log 则在用户与管理交互过程中动态写入。

6 推荐算法设计

本项目共实现两类推荐模型。第一类为传统方法，包括作为参照的 6 类基线（全局均值、用户均值、物品均值、正则化偏置、随机评分、最受欢迎）以及核心算法 UserCF、ItemCF、SVD 与混合模型；这些模型均通过解析解、邻域加权或矩阵分解直接求解，不依赖梯度下降式的迭代训练，因此不涉及损失函数、优化器与训练轮次等概念。第二类为深度学习方法，即基于 PyTorch 实现的神经协同过滤（NeuralCF，src/neural_cf.py），通过嵌入向量与多层感知机对用户 - 电影交互进行端到端训练，纳入统一的实验对比。

在深度学习框架的选型上，本项目主体算法（CF/MF）属于可解析或可分解求解的传统模型，无需 GPU 训练，使用 scikit-learn/NumPy 即可高效实现且具有更好的可复现性；而对于需要端到端梯度训练的 NeuralCF，本项目优先选用 PyTorch 框架，原因在于 PyTorch 的动态计算图便于调试、自动求导接口简洁、生态成熟，且与课程所采用的技术栈一致。综合考虑，传统模型使用 scikit-learn 实现、深度模型使用 PyTorch 实现的混合技术选型，既保证了传统基线的高效与可复现，又满足了对深度学习方法的探索需求。

6.1 基于用户的协同过滤（User-Based CF）

UserCF 的基本思想是“兴趣相近的用户会喜欢相似的电影”。系统在训练集评分矩阵上计算用户两两之间的相似度（余弦相似度/皮尔逊相关系数），对目标用户选取相似度最高的 K 个近邻（ $K_NEIGHBORS=20$ ），以近邻对目标电影评分的相似度加权平均作为预测评分。其优点是直观、可解释性强；局限在于用户规模增大时相似度矩阵计算成本高，且在稀疏数据下用户间共同评分较少，相似度估计方差较大。

6.2 基于物品的协同过滤（Item-Based CF）

ItemCF 将相似度建立在电影之间：若两部电影被大量用户给出相近的评分，则认为它们相似。系统为目标用户已评分的电影寻找相似电影并加权汇总其评分，从而生成推荐。相比 UserCF，物品相似度通常随时间更稳定，且更便于离线预计算，更适合电影数量相对固定的场景；其局限在于对新上架、评分稀少的电影同样存在冷启动问题。

6.3 基于偏置残差的 SVD 矩阵分解

SVD 模型采用两阶段建模：第一阶段用带 L2 正则项的偏置基线拟合“全局均值 + 用户偏置 + 物品偏置”，刻画评分的整体水平；第二阶段对评分残差矩阵（真实评分减去偏置预测）做 TruncatedSVD 低秩分解（ $SVD_N_COMPONENTS=50$ ），得到用户与物品的隐因子向量，最终预测值为“偏置预测 + 用户隐向量与物品隐向量的内积”，并裁剪到 $[1, 5]$ 区间。相比直接对原始稀疏矩阵分解，先扣除偏置再分解残差能够降低稀疏性带来的不稳定，更好地刻画用户与电影之间的潜在交互，因此在评分预测任务上取得了本项目最优的 RMSE/MAE。

6.4 基线模型与元数据混合模型

6 类基线为各模型提供了统一的参照系：全局/用户/物品均值直接取相应子集评分的算术平均；正则化偏置基线通过带正则项的最小二乘近似求解用户与物品偏置，敏感性分析确定正则化系数

reg=1.0 为最优；随机评分在训练集评分范围内均匀采样，最受欢迎按评分次数排序，二者分别作为质量下界与流行度参照。元数据混合模型（Hybrid，src/hybrid_model.py）以用户/物品偏置、用户对各类型电影的历史均分、电影类型与年份等元数据特征为输入训练回归模型，对评分稀少或新上架的电影更为鲁棒，是系统应对冷启动的算法手段之一。

6.5 神经协同过滤（NeuralCF）

NeuralCF 是本项目实现的深度学习推荐模型，基于 PyTorch 构建。其网络结构（src/neural_cf.py 中的 _NCFNetwork）由用户嵌入层、电影嵌入层与多层感知机（MLP）组成：每个用户与电影分别映射为 32 维的嵌入向量，将两者拼接后送入隐藏层维度为（64，32）的 MLP（各层后接 ReLU 激活与 Dropout，丢弃率 0.1），最终由输出层回归预测评分。相比线性的矩阵分解，MLP 能够建模用户与电影之间的非线性交互。

该模型的训练要素如下：

- 损失函数：采用均方误差损失（MSELoss），即最小化预测评分与真实评分之间的平方误差，与 RMSE/MAE 的评分预测目标一致；
- 优化器：采用 AdamW 优化器（学习率 1e-3，权重衰减 weight_decay=1e-5），在自适应学习率的基础上引入解耦权重衰减以缓解过拟合；
- 训练策略：批量大小 batch_size=512，训练 20 个 epoch，按 batch 进行小批量随机梯度更新；通过 Dropout 与权重衰减进行正则化；固定随机种子（torch.manual_seed=42、DataLoader 生成器种子=42）以保证训练过程可复现；
- 预测处理：对训练集中未出现的用户/电影回退为全局均值，所有预测评分裁剪至 [1, 5] 区间，与其他模型保持一致的评估口径。

训练过程中每个 epoch 的平均训练损失被记录于模型的 history_ 属性，用于绘制训练损失曲线（详见第 8.6 节训练过程可视化）。

6.6 推荐理由生成

为提升推荐结果的可解释性，在线推荐服务（src/recsys_service.py）会根据所用算法与电影类型，为推荐结果生成自然语言推荐理由：

- 当采用 UserCF 时，推荐理由强调“与你评分行为相似的用户对该电影给出了较高评价”；
- 当采用 SVD 时，推荐理由强调“潜在因子模式显示你对该电影具有较高的个性化偏好”；
- 当基于物品/类型相似度时，推荐理由强调“你评分过的电影中包含相似的某类型影片”；
- 对新用户或缺乏历史行为的场景，则结合注册时选择的偏好类型与热门/高分电影给出推荐理由。

这种“算法 + 理由”的设计，使推荐结果不再是黑箱输出，而是能够向用户解释“为什么推荐这部电影”，与第 7 章“个性化推荐”“相似电影”等界面相互呼应。

7 系统实现

本章基于实际运行截图，对 FilmTrace 系统的用户端与管理员端功能逐一展示。每个模块均说明其功能、所用数据及交互方式，并阐明其在整个推荐系统中的作用。系统界面采用深色影院主题，整体风格统一。

7.1 系统欢迎页与角色入口

图所示为 FilmTrace 系统的欢迎页，是用户进入系统的统一入口。页面顶部突出展示系统名称与“电影推荐系统”主题，并以卡片形式展示 MovieLens 100K 的核心规模信息（943 名用户、1,682 部电影、100,000 条评分）以及 UserCF、ItemCF、SVD 等所采用的核心算法标签，使访问者在进入系统前即可了解系统的数据规模与技术路线。页面同时提供用户登录/注册与管理员入口，实现用户端与管理员端的角色分流。

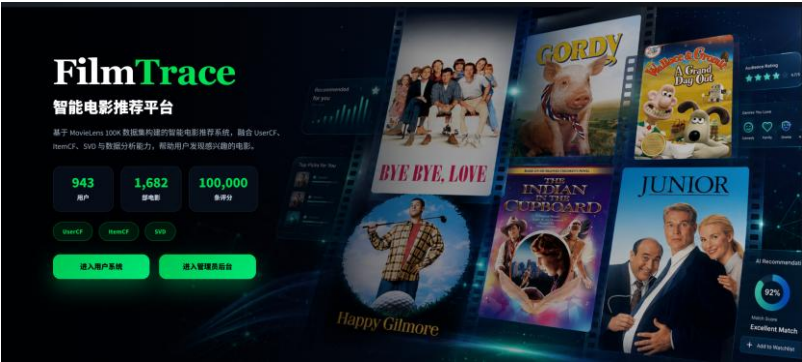


图 5 FilmTrace 系统欢迎页

7.2 用户注册、登录与偏好选择

图所示为用户登录界面，用户通过用户名与密码登录系统。登录信息经 `src/accounts.py` 校验，账号数据存储于 SQLite 的 `accounts` 表，密码以哈希形式（`password_hash`）保存，保证账号安全。登录成功后，系统通过 `session_state` 记录当前账号与角色信息，并据此渲染对应的用户端或管理员端导航。



图 6 用户登录界面

图所示为用户注册与偏好选择界面。新用户注册时除填写账号信息外，还需从电影类型列表

勾选感兴趣的类型（如 Action、Comedy、Drama 等）。该偏好数据被写入 user_preferences 表的 genres 字段，并在用户尚无评分历史时作为生成初始推荐候选的主要依据，是系统应对“新用户冷启动问题”的关键设计。

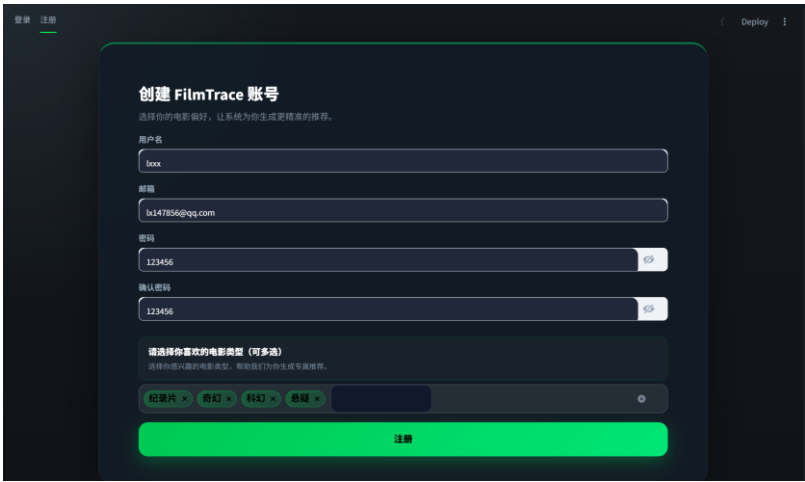


图 7 用户注册与偏好选择界面

结合第 5 章数据库设计可知，该页面提交的数据最终落入 accounts 与 user_preferences 两张表，二者通过 account_id 关联。

7.3 用户端首页、电影库与电影详情

图所示为用户登录后的用户端首页，页面以导航形式整合了“为你推荐”“电影库”“我的收藏”“评分记录”“我的画像”等主要功能入口，是用户完成个性化推荐工作流的起点。

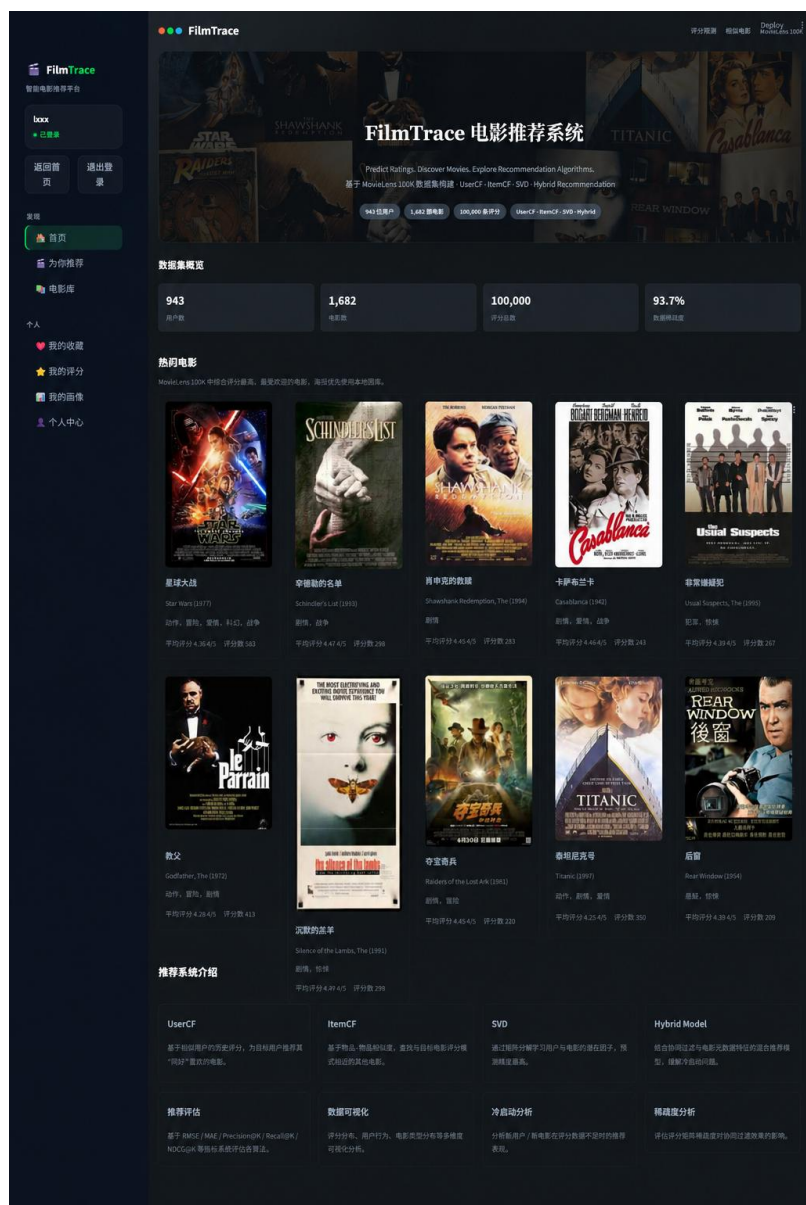


图 8 用户端首页界面

图所示为用户端电影库页面，以海报卡片形式批量展示 MovieLens 100K 中的电影。每张卡片包含由 src/posters.py 从本地“电影照片/”目录匹配得到的海报、片名、上映年份与类型标签，并支持按类型、年代等条件筛选检索。用户可在此浏览全部电影并进行评分、收藏，是数据集内容的主要浏览入口。

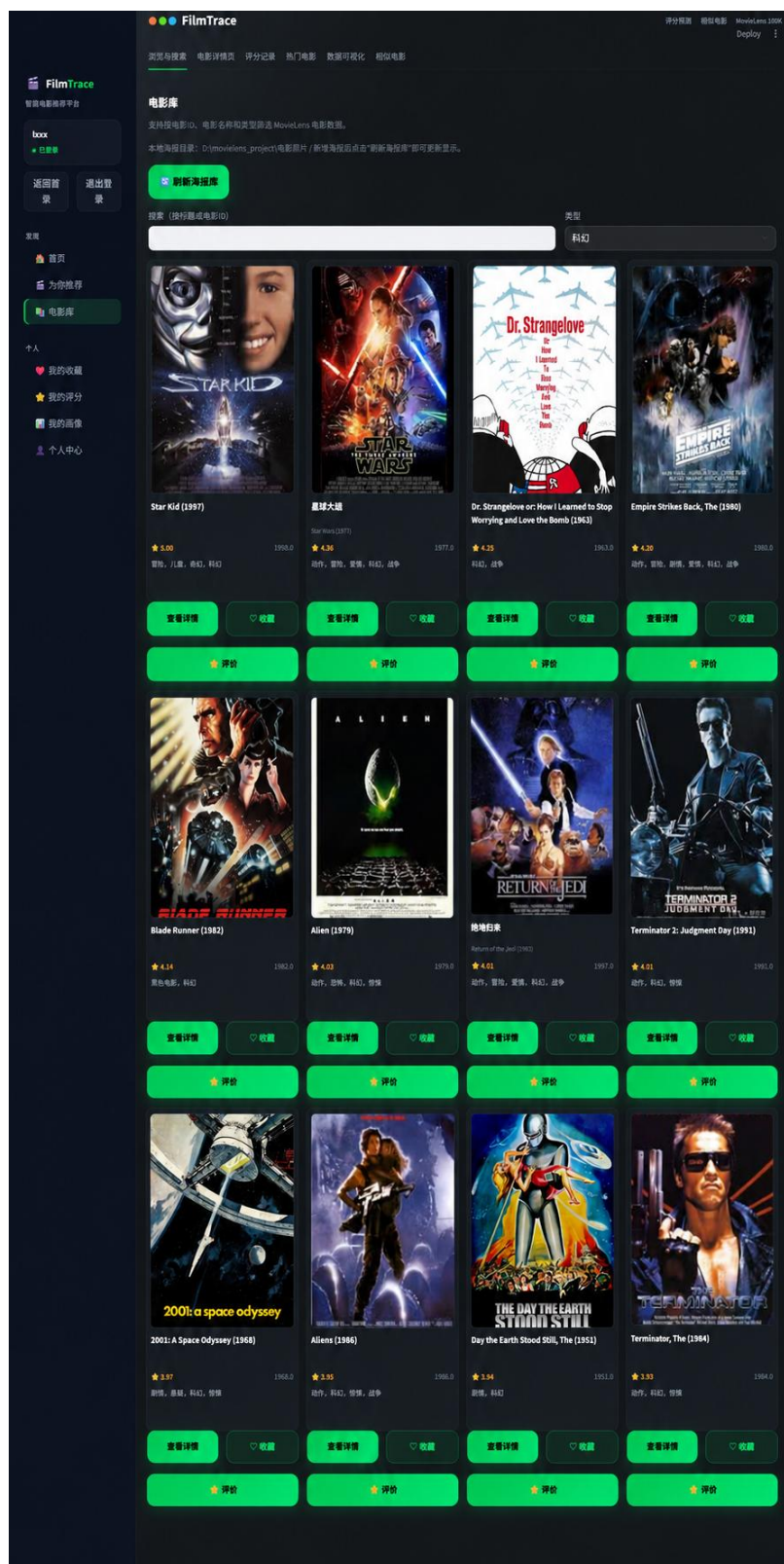


图 9 用户端电影库界面

图所示为单部电影的详情页，呈现电影的片名、上映年份、类型标签、本地海报及（如有）IMDb 链接等元数据信息，为用户提供比列表视图更完整的电影信息，辅助其做出是否评分、收藏或观看的决策。

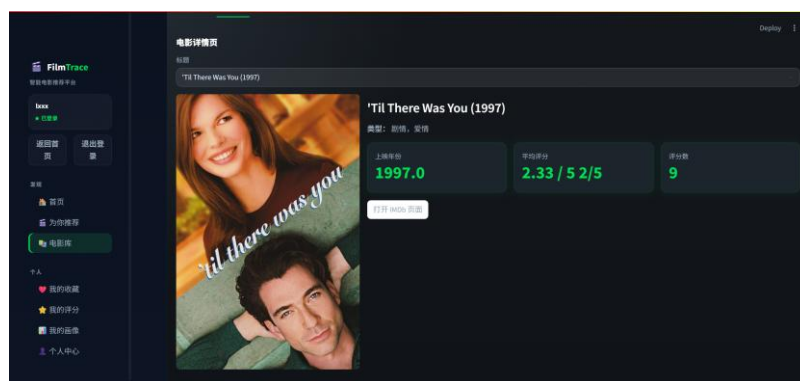


图 10 电影详情展示界面

图所示为从电影详情页跳转的 IMDb 信息展示页面，作为对本地电影元数据的外部信息补充，为用户提供该电影在 IMDb 平台上的参考信息，进一步辅助观影决策。

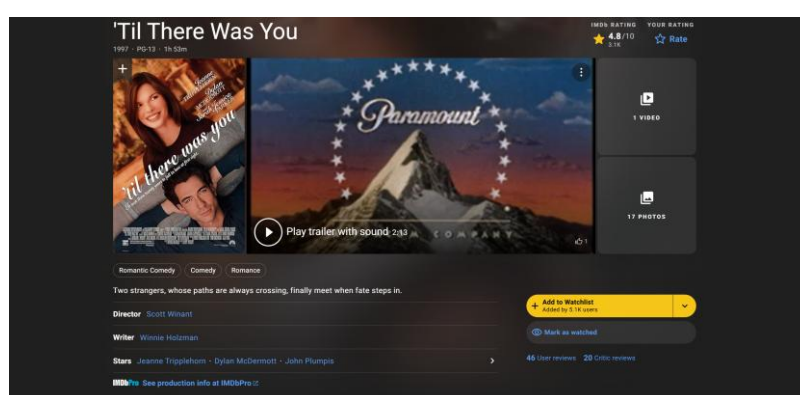


图 11 电影 IMDb 信息展示界面

7.4 个性化推荐与相似电影推荐

图所示为“为你推荐”页面的个性化推荐结果。系统结合用户的历史评分与注册偏好类型，调用第 6 章所述的协同过滤与偏置/SVD 模型计算候选电影的预测评分，并以海报卡片形式展示推荐列表，每张卡片标注预测评分（或近似评分区间）及推荐理由。该页面是系统核心推荐能力的直接呈现。

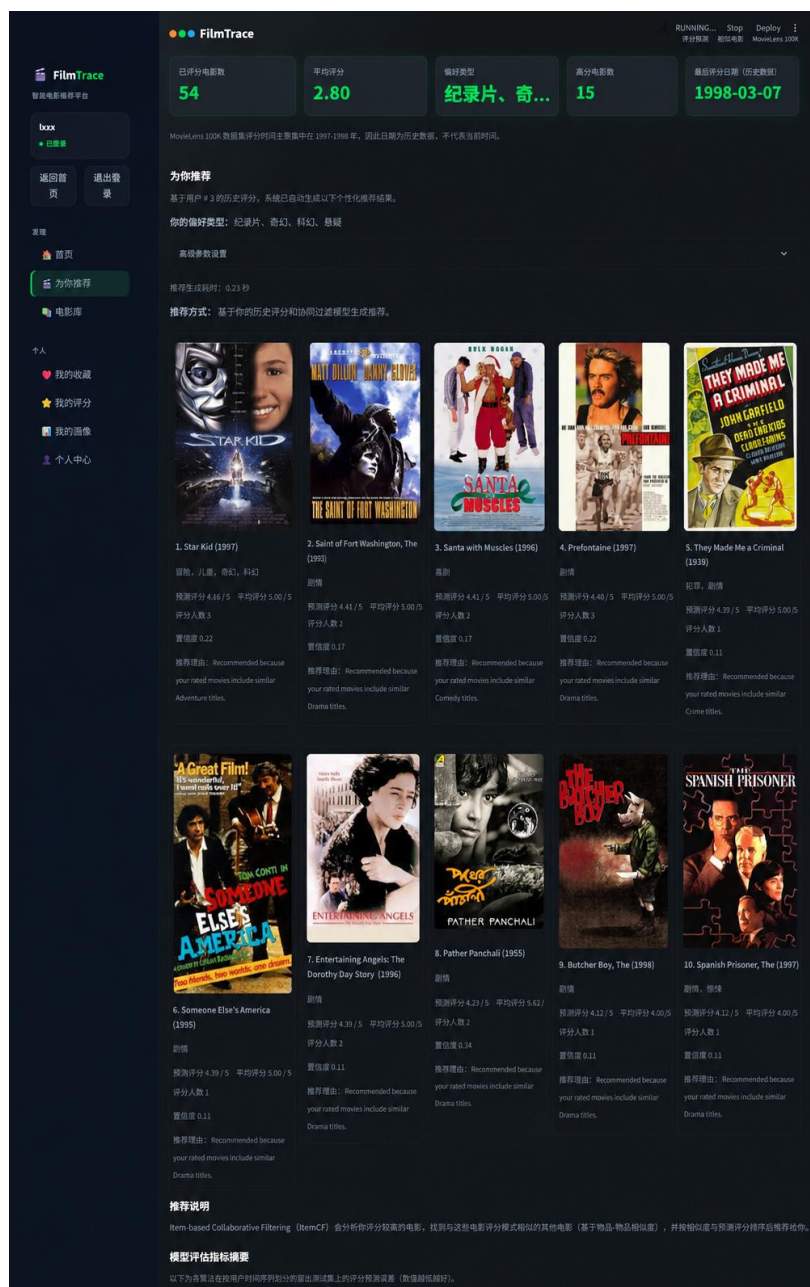


图 12 个性化推荐结果展示界面

需要说明的是，卡片所示预测评分区间是基于离线残差分位数给出的近似参考，而非严格置信区间。

图所示为“相似电影”推荐页面。系统基于电影类型相似度，为用户当前浏览的电影推荐风格相近的其他电影，对应 `src/recsys_service.py` 中基于类型相似度的推荐逻辑，为用户提供“继续探索”的延伸路径，是基于内容（电影元数据）的推荐能力在前端的体现。

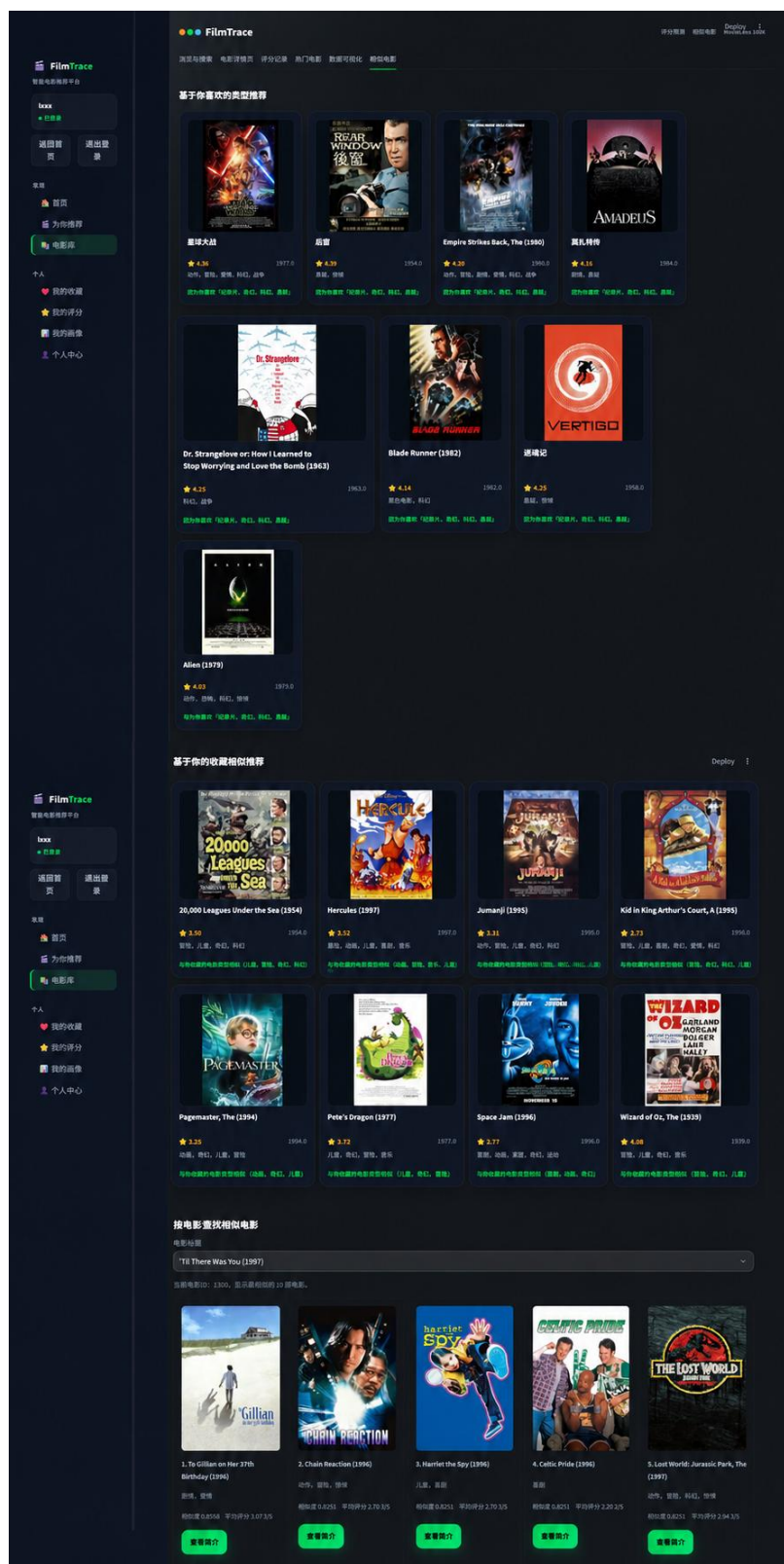


图 13 相似电影推荐界面

图所示为电影库的“热门电影”视图，按电影在数据集中被评分的次数（流行度）排序展示。该视图与第 6 章“最受欢迎（Most Popular）”基线在排序逻辑上一致，也是第 9 章讨论“流行度基线在 Top-N 指标上优于个性化模型”这一现象时的直观印证。

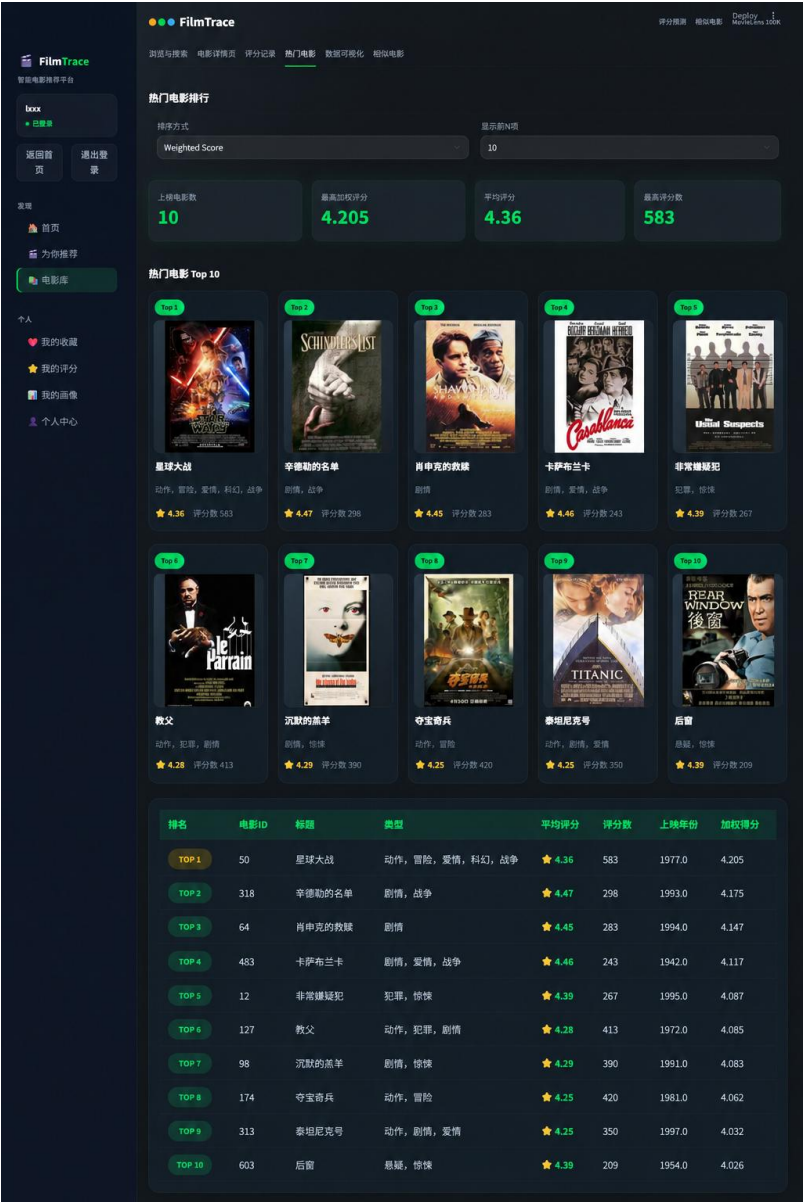


图 14 电影库热门电影界面

7.5 用户评分、评价与收藏管理

图所示为用户评分记录与偏好分析界面，汇总用户对各电影的历史评分并提供评分次数、平均分等统计信息。该页面数据直接来源于 SQLite 的 ratings 表（user_id 对应当前账号关联的 MovieLens 用户），是个性化推荐的重要输入。

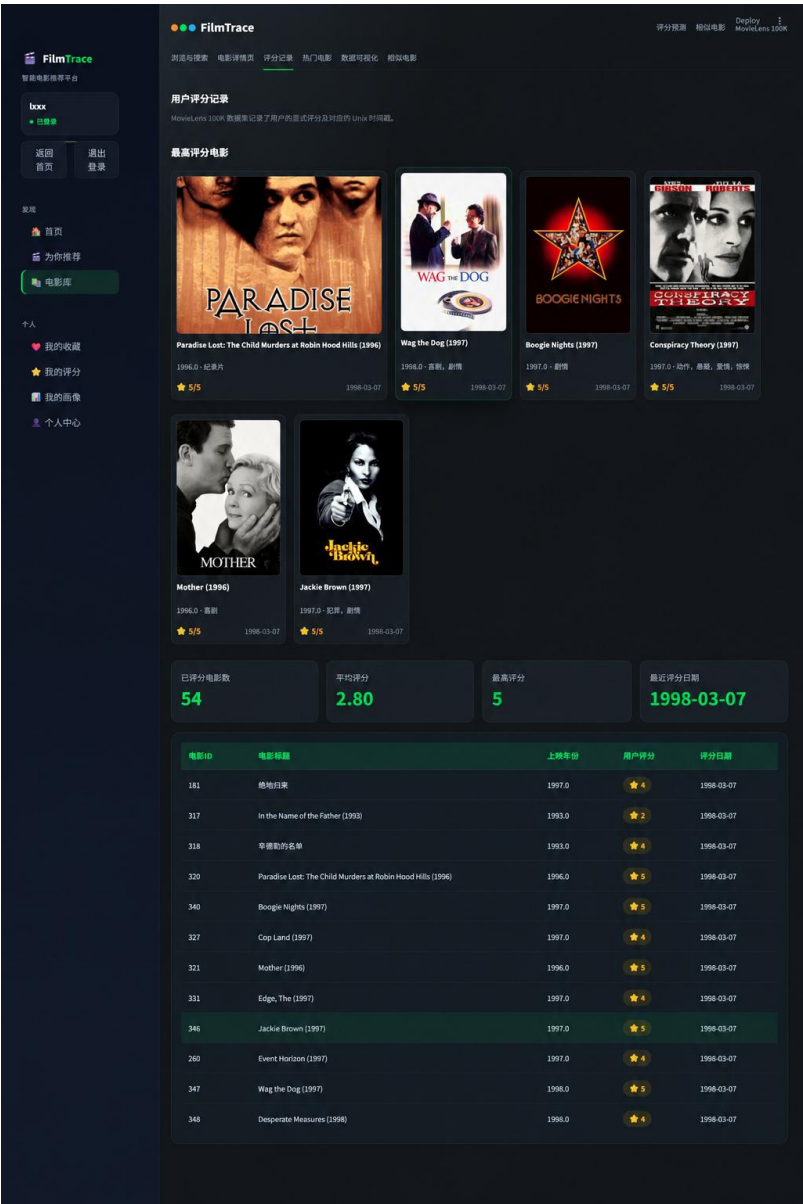


图 15 用户评分记录与偏好分析界面

图所示为“我的评价”界面，展示用户撰写的文字评价。评价内容写入 ratings 表的 review 字段，与评分一并构成用户对电影的显式反馈。

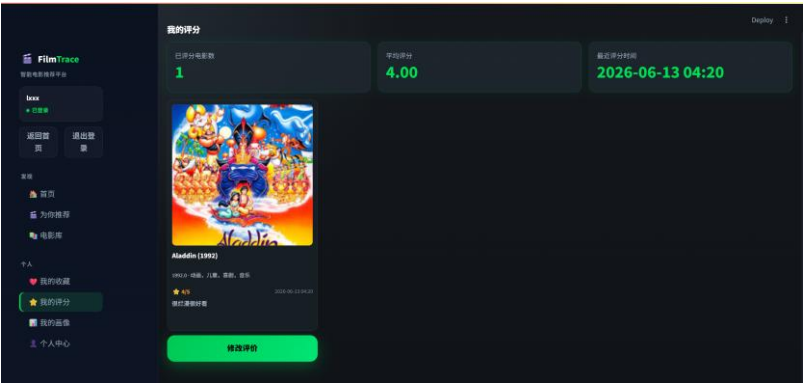


图 16 我的评价界面

图所示为用户收藏管理界面，展示用户收藏的电影。收藏数据存储在 favorites 表（account_id 与 movie_id 联合主键），用户可在此查看与管理收藏，收藏行为也可作为推荐与画像分析的辅助信号。



图 17 用户收藏管理界面

图所示为用户个人中心界面，集中展示账号信息与个人相关数据入口，便于用户管理自身账号与偏好设置。

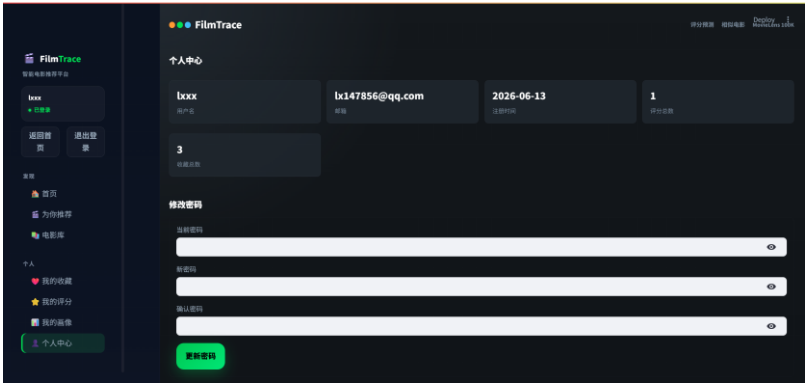


图 18 用户个人中心界面

7.6 用户画像与数据可视化

图所示为用户电影画像分析界面。系统基于用户的评分与收藏行为，统计其偏好类型、年代分布、评分习惯等，生成个性化的“电影画像”，并写入 user_profiles 表（top_genres、year_range、preferred_min_rating 等）。该模块将分散的用户行为数据转化为直观的画像，提升了系统的可解释性。



图 19 用户电影画像分析界面

图所示为用户端数据可视化分析界面，借助 Plotly/Matplotlib 将评分分布、类型偏好、推荐结果分布等以交互图表形式呈现，帮助用户从整体上理解自身观影口味与平台数据特征。



图 20 用户端数据可视化分析界面

7.7 管理员后台功能实现

图所示为管理员后台首页。管理员登录后进入独立的后台界面，可访问电影库管理、用户管理、评分记录、算法配置、模型评估、系统统计等功能，所有写操作均通过 `src/db.py` 执行并记录至 `admin_audit_log`，实现权限隔离与操作留痕。

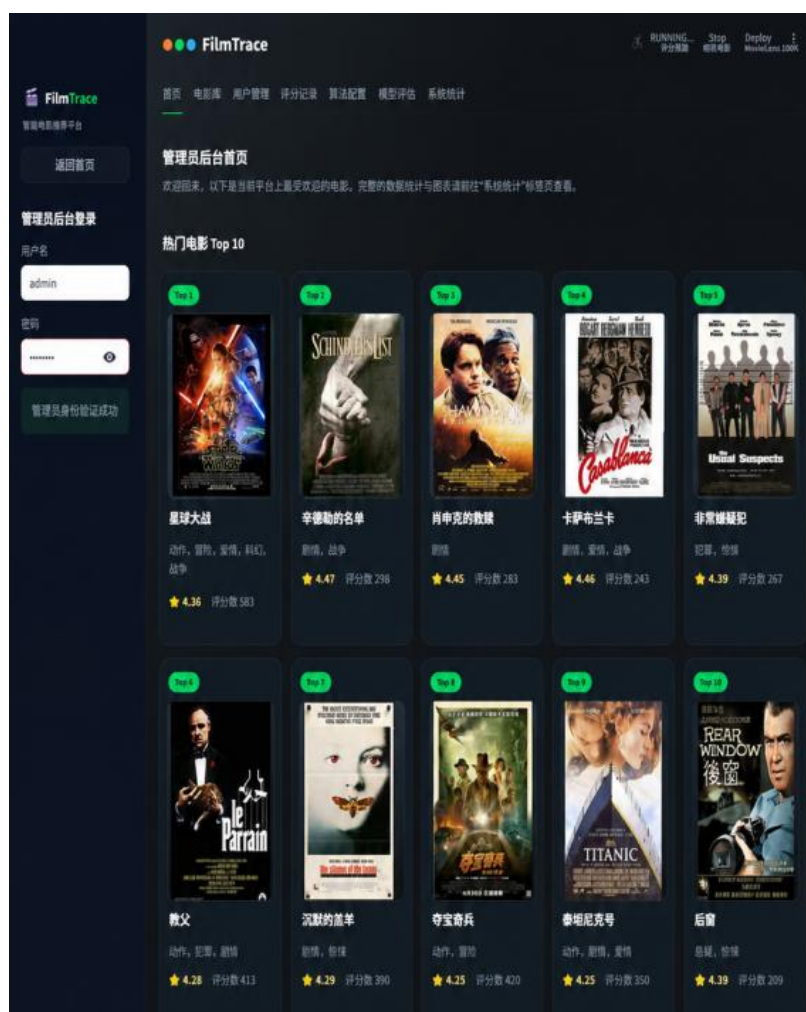


图 21 管理员后台首页

图所示为管理员电影数据管理界面，支持对 movies 表中电影信息的查询与管理（CRUD），便于维护电影库内容。

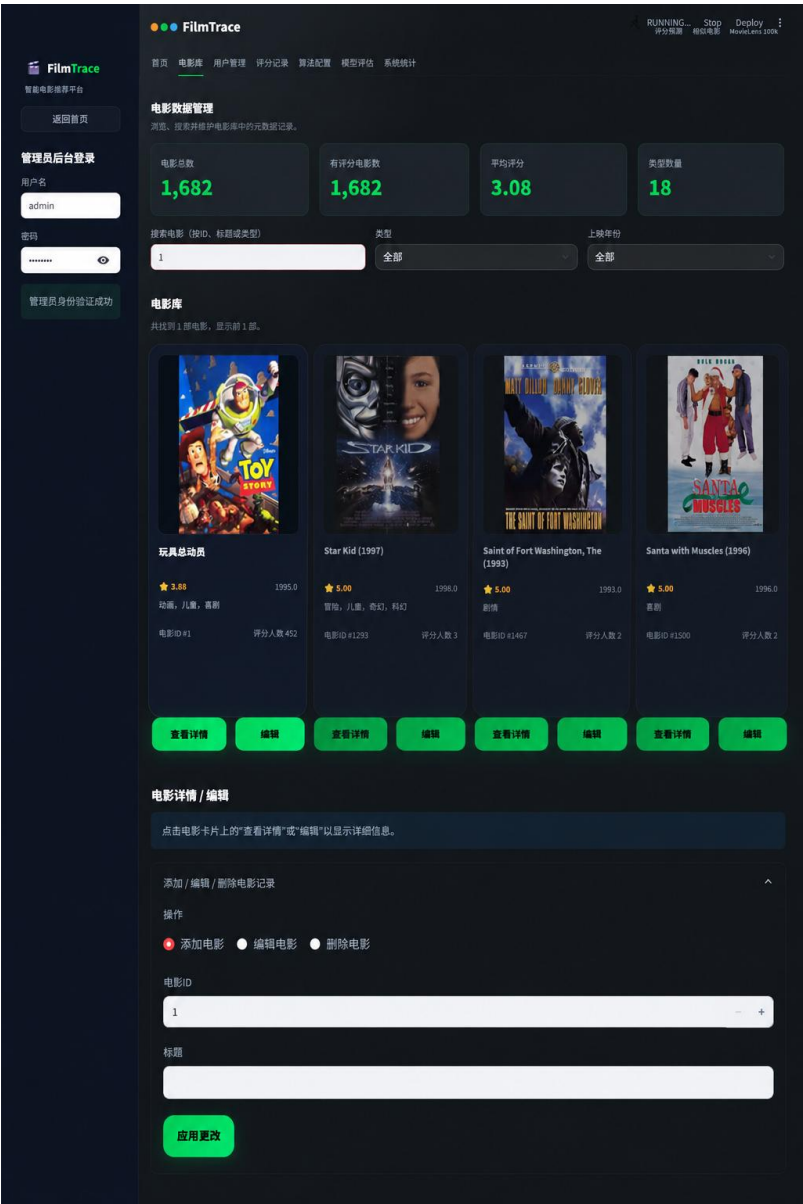


图 22 管理员电影数据管理界面

图所示为管理员用户数据管理界面，支持对系统账号与 MovieLens 用户数据的查询、检索与管理，便于运营方掌握用户规模与活跃情况。

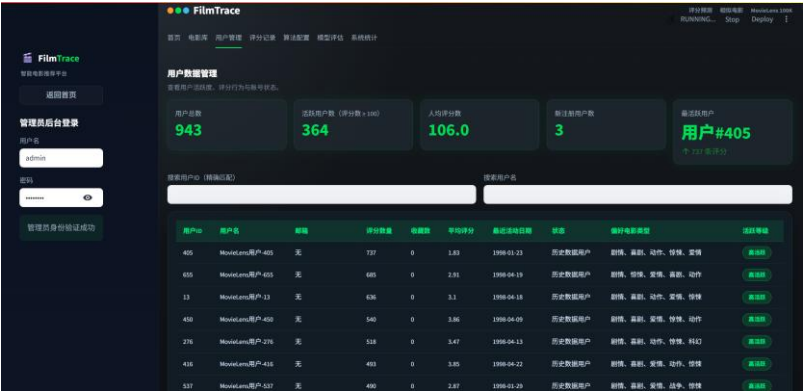


图 23 管理员用户管理界面

图所示为管理员评分记录管理界面，集中展示与管理 ratings 表中的评分数据，为数据质量核查与统计分析提供入口。

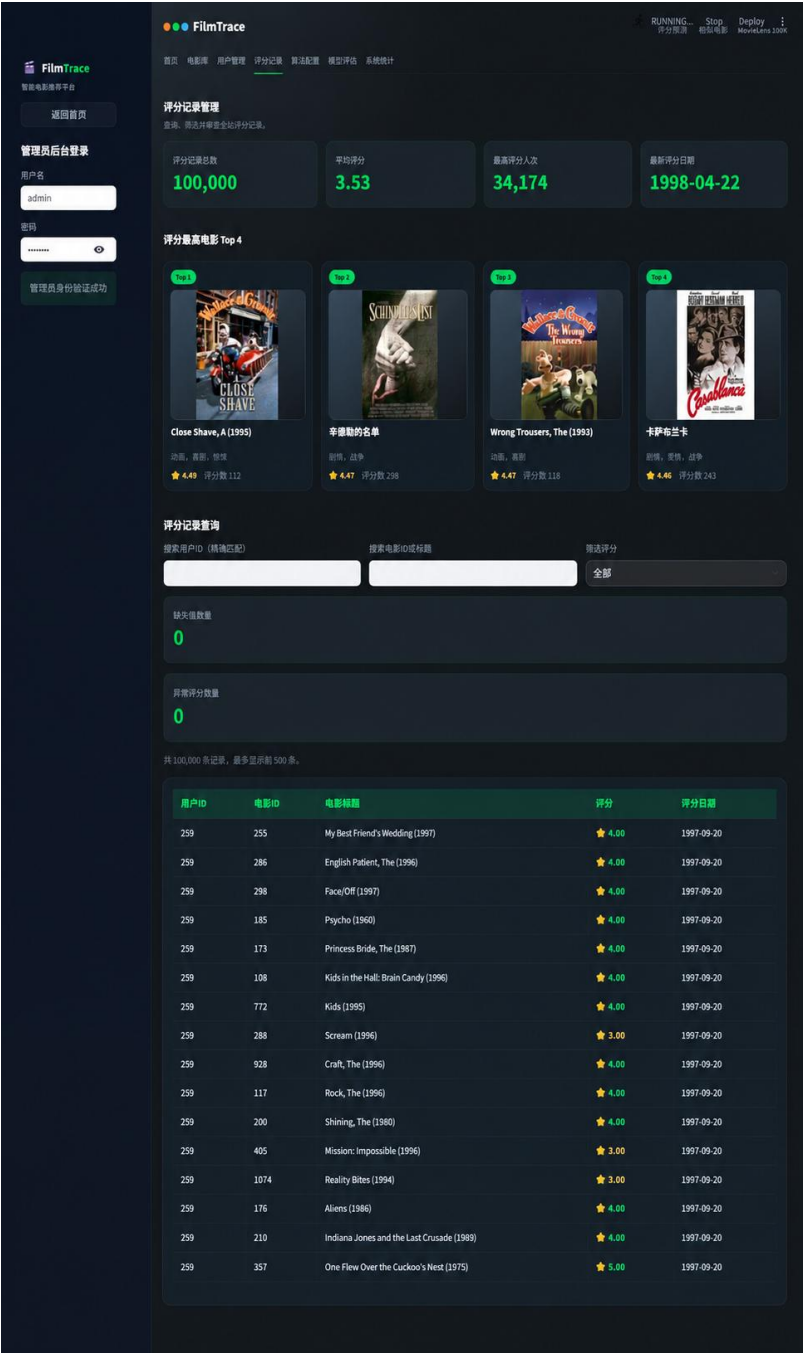


图 24 管理员评分记录管理界面

图所示为推荐算法配置界面，管理员可在此选择推荐算法并调整关键参数（如近邻数 K、SVD 隐因子数等），实现对推荐策略的灵活配置。

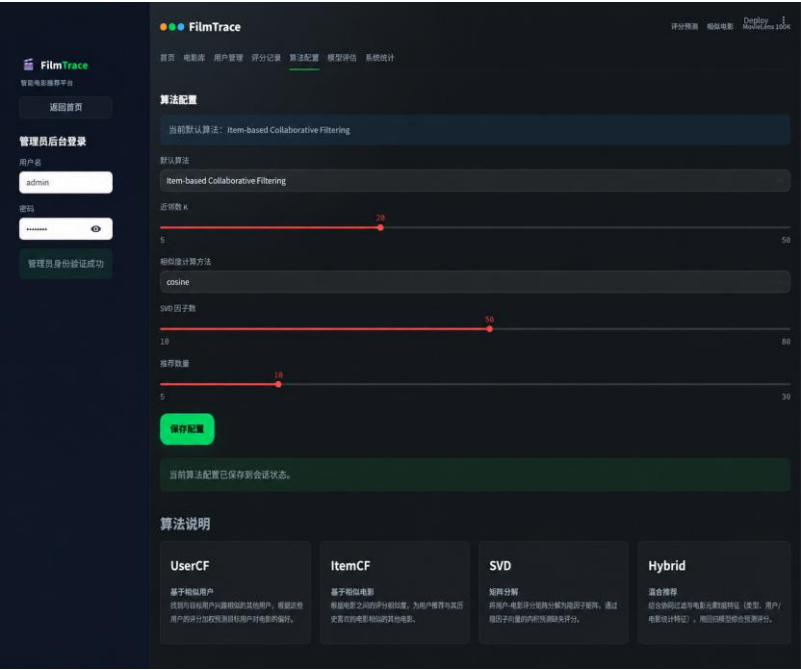


图 25 推荐算法配置界面

图所示为推荐模型评估界面，管理员可在此查看各模型在 MAE、RMSE 及 Top-N 指标上的评估结果，将第 8、9 章的离线评估结论以可视化形式呈现，支撑算法选型决策。

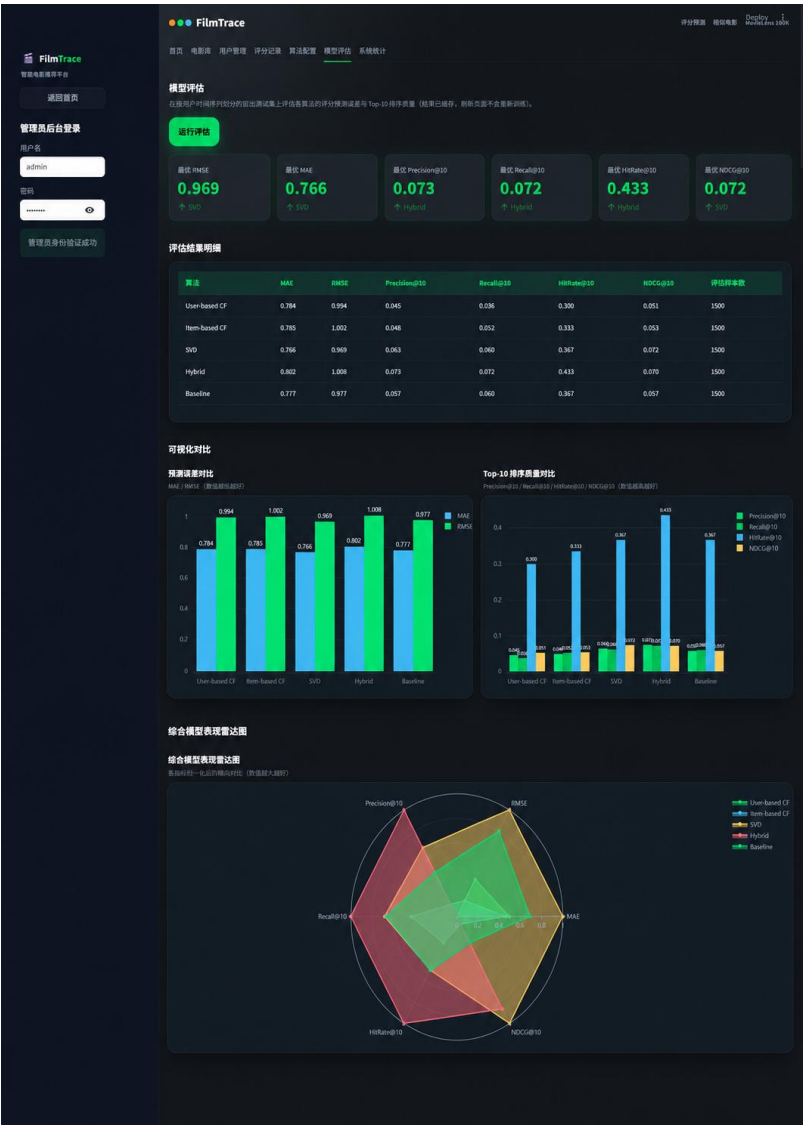


图 26 推荐模型评估界面

图所示为系统统计与数据分析界面，从整体上汇总用户、电影、评分等关键指标的统计信息，为运营方提供系统运行状况的总体视图。



8 实验设计与模型评估

8.1 评估协议与指标

实验在按用户时间序列划分的训练/测试集上进行：每位用户的较早评分用于训练，较晚评分用于测试，所有相似度矩阵、用户/物品均值、偏置项、混合特征与 SVD 因子均仅在训练集上拟合，RMSE/MAE 仅在留出的测试评分上计算，以避免信息泄漏。Top-N 评估针对每位用户的留出评分进行，采用全目录候选排序，默认以测试集真实评分 ≥ 4.0 作为相关性判定，K 取 10，参与 Top-N 评估的用户数为 60。各评估指标定义如表 6 所示。

表 6 评估指标定义

指标	定义	评估角度
RMSE	均方根误差，对较大误差惩罚更重	评分预测精度
MAE	平均绝对误差	评分预测精度
Precision@K	Top-K 列表中相关物品（评分 $\geq$ 阈值）的比例	Top-N 排序质量
Recall@K	相关物品被召回进 Top-K 的比例	Top-N 排序质量
HitRate@K	Top-K 中至少命中一个相关物品的用户比例	Top-N 命中能力
NDCG@K	考虑相关物品排序位置的归一化折损累积增益	Top-N 排序质量
Catalog Coverage	推荐结果覆盖的电影占整个电影库的比例	推荐多样性/覆盖

8.2 实验设置与参数

主要实验参数如表 7 所示，所有随机过程固定 random_state=42 以保证可复现。

表 7 主要实验参数设置

参数	取值	说明
random_state	42	全局随机种子
test_size	0.2	测试集比例（按用户时间序列划分）
K_NEIGHBORS	20	UserCF/ItemCF 近邻数（敏感性分析范围 5 - 50）
SVD_N_COMPONENTS	50	SVD 隐因子数量
偏置正则化系数 reg	1.0	正则化偏置基线最优值（敏感性分析确定）
相关性阈值	4.0	Top-N 相关判定（敏感性分析含 3.0/3.5/4.0）
Top-N 的 K	10	推荐列表长度

8.3 模型评估结果

图 28 与图 29 分别给出 9 个模型在评分预测指标（RMSE/MAE）与 Top-N 排序指标（Precision@10/Recall@10/NDCG@10）上的对比；表 8 汇总了关键评估结果（数据来源：执行后的 notebook，notebooks/01_movielens_recommendation_executed.ipynb，为本项目结果的“真值”来源）。



图 28 各模型评分预测误差 (RMSE/MAE) 对比

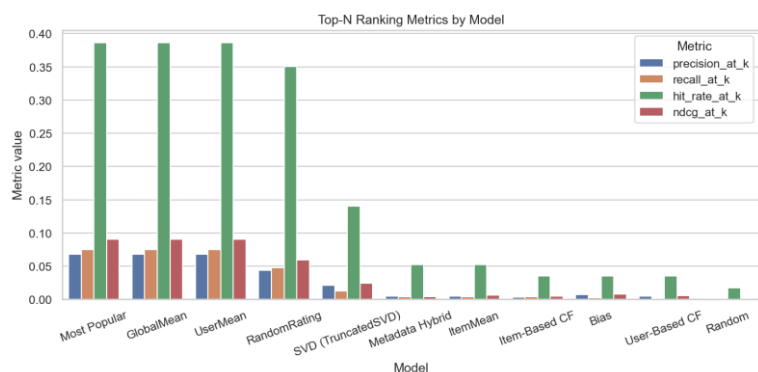


图 29 各模型 Top-N 推荐指标对比

表 8 关键评估结果汇总

指标	最优模型	数值
RMSE	SVD（偏置残差 TruncatedSVD）	0.9826
MAE	SVD（偏置残差 TruncatedSVD）	0.7763
RMSE / MAE（深度模型）	NeuralCF（PyTorch）	1.0080 / 0.8003
Precision@10	Most Popular / GlobalMean / UserMean	0.0684
Recall@10	Most Popular / GlobalMean / UserMean	0.0752
NDCG@10	Most Popular / GlobalMean / UserMean	0.0908
Catalog Coverage（随机基线）	Random	0.3014
Catalog Coverage（最优非随机）	User-Based CF	0.0916

8.4 相关性阈值敏感性分析

由于 Top-N 指标对相关性阈值较为敏感，实验对阈值 3.0/3.5/4.0 分别评估，结果如表 9 所示。

表 9 Top-N 相关性阈值敏感性分析

阈值	最优 Precision@10	最优 Recall@10	最优 NDCG@10
3.0	0.0883	0.0691	0.1113
3.5	0.0684	0.0752	0.0908
4.0	0.0684	0.0752	0.0908

结果显示，在三档阈值下，取得最优 Top-N 指标的始终是 Most Popular / GlobalMean / UserMean 等流行度类基线，说明该现象在不同阈值下稳定存在，而非阈值选择所致。

8.5 超参数敏感性分析

图 30 给出 UserCF/ItemCF 在近邻数 K (5 - 50) 变化时的 RMSE 趋势，图 31 给出正则化偏置基线在不同正则化系数下的 RMSE 趋势（据此确定 $\text{reg}=1.0$ 为最优）。由于本项目模型不依赖梯度迭代训练，故以此类超参数敏感性曲线替代深度学习中的“训练过程（损失）曲线”。

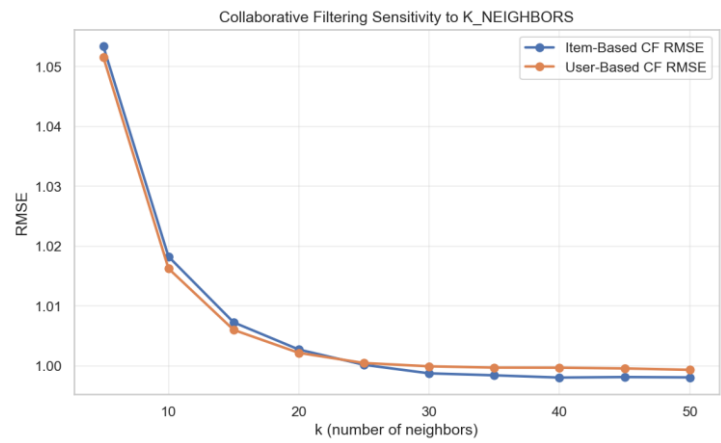


图 30 UserCF/ItemCF 近邻数 K 对 RMSE 的影响

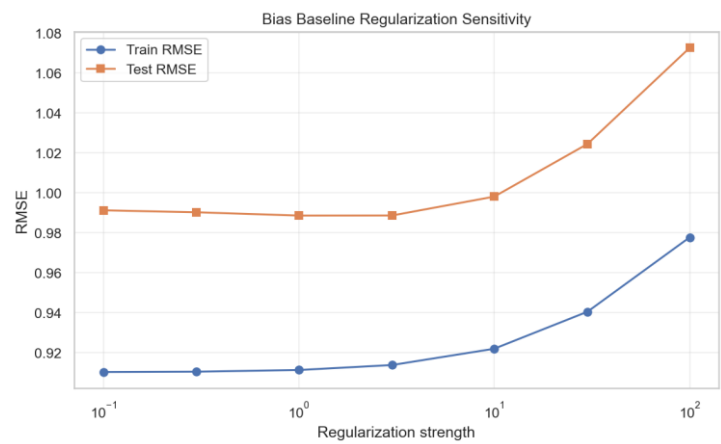


图 31 偏置基线正则化系数对 RMSE 的影响

8.6 NeuralCF 训练过程可视化

与传统模型不同，NeuralCF 通过小批量随机梯度下降进行端到端训练。为观察其收敛情况，记录了训练过程中每个 epoch 的平均训练损失（MSE），并绘制训练损失曲线，如图 32 所示。模型在按用户时间序列划分的训练集（80,000 条评分）上训练 20 个 epoch，采用 MSELoss 损失函数与 AdamW 优化器（学习率 $1e-3$ 、权重衰减 $1e-5$ 、批量 512）。

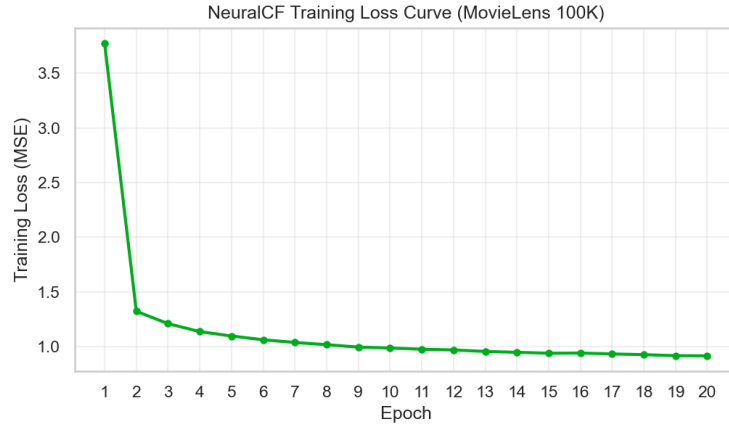


图 32 NeuralCF 训练损失 (MSE) 随 epoch 变化曲线

从图 32 可见，训练损失在第 1 个 epoch 后由约 3.77 迅速下降至约 1.32，随后平稳收敛，至第 20 个 epoch 降至约 0.91，曲线无明显震荡或回升，说明在当前学习率与正则化设置下训练过程稳定、未出现明显过拟合。最终 NeuralCF 在测试集上取得 RMSE=1.0080、MAE=0.8003，与传统协同过滤处于同一量级，但略逊于经过偏置-残差建模的 SVD (RMSE=0.9826)。这一结果与已有研究中“在 MovieLens 等中小规模、稀疏的显式评分数据集上，结构简单的神经推荐模型未必优于精心设计的矩阵分解”的观察一致，也说明深度模型的优势更依赖于大规模数据与更复杂的结构设计。

9 结果分析

9.1 定量结果分析

从评分预测角度看，基于偏置残差的 TruncatedSVD 取得了最优的 RMSE (0.9826) 与 MAE (0.7763)，优于其余 8 个模型，验证了“先拟合偏置、再分解残差”这一两阶段建模思路在稀疏评分矩阵上的有效性。这表明在评分预测这一回归任务上，矩阵分解能够比纯邻域方法更充分地利用用户与电影之间的潜在交互信号。

然而从 Top-N 排序角度看，结果出现了值得深究的反直觉现象：在全目录离线评估协议下，Most Popular、GlobalMean、UserMean 等流行度类基线在 Precision@10、Recall@10、NDCG@10 上反而领先于 UserCF、ItemCF、SVD、Hybrid 等个性化模型，且该现象在 3.0/3.5/4.0 三档阈值下一致出现。

9.2 错误案例与原因分析

结合 README 中的实验记录与相关研究，本项目认为上述现象主要由评估协议与数据特性共同导致：

- 评估协议层面：离线 Top-N 将“未评分电影”视为“未知”而非“真实负样本”，在全目录候选排序中，每部未观看电影都参与竞争 Top-10，但仅有少量留出评分能计为正样本，这会系统性地压低 Precision/Recall，并使流行度基线因“热门即高命中概率”而占优（与 Cremonesi 等人的结论一致[5]）；
- 数据稀疏层面：MovieLens 100K 评分矩阵稀疏度高达 93.7%，用户-用户、物品-物品之间可用于估计可靠邻域的共同评分较少，导致 UserCF/ItemCF 的相似度估计方差较大，Top-N 排序信号不足；
- 评估规模层面：参与 Top-N 评估的用户为 60 名，每位用户的留出正样本较少，进一步放大了排序指标的波动。

因此，本报告将这一结果解读为“在该离线协议下，个性化排序器尚未提取出足够的排序信号”的警示信号，而非“个性化算法完全失效”。更接近生产实践的做法，是将流行度先验、偏置项、隐因子与重排序相结合，而非依赖单一的最近邻协同过滤。

9.3 覆盖率与多样性分析

图 33 给出各模型推荐结果的多样性与新颖性对比。结合覆盖率结果可见：Random 基线的目录覆盖率最高 (0.3014) 但推荐质量最低，体现了“覆盖率高不等于质量好”；在非随机模型中，User-Based CF 的覆盖率最高 (0.0916)，说明其推荐结果在电影库中的分布相对其他个性化模型更广，但与 Random 相比仍有明显差距。这清晰地展示了推荐系统中“准确率—覆盖率/多样性”之间的典型权衡。

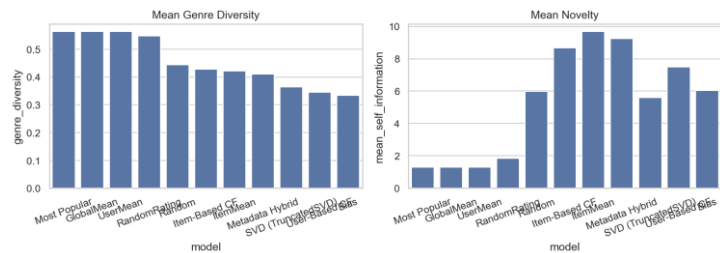


图 33 各模型多样性与新颖性对比

9.4 冷启动与定性结果

针对新用户冷启动问题，系统通过注册时的“偏好类型选择”结合元数据混合模型为新用户生成初始推荐候选，在缺乏历史评分时仍能给出与用户偏好类型相关的推荐，缓解了纯协同过滤在冷启动场景下的失效。从定性结果看（参见第 7.4 节个性化推荐界面），系统能够结合用户历史评分与偏好类型给出带预测评分与推荐理由的推荐列表，推荐结果与用户画像（第 7.6 节）在偏好类型上保持一致，体现了较好的可解释性。

9.5 局限性小结

- MovieLens 100K 规模较小、年代较早，实验结论不宜直接外推到大规模实时业务场景；
- 全目录离线 Top-N 协议将“未评分”视为“不相关”，可能低估个性化模型的真实排序能力，且与采样负例基准不可直接比较；
- 纯协同过滤对新用户/新电影的冷启动仍有固有局限，本项目仅做了缓解而非根本解决；
- 评估均基于离线数据，未引入在线 A/B 测试与真实点击日志。

10 项目测试与运行说明

为保证项目的可复现性，本节给出从环境准备到系统运行的完整步骤。

10.1 运行环境与依赖

项目基于 Python 实现，主要依赖 pandas、NumPy、scikit-learn、Streamlit、Plotly/Matplotlib 等库；可选的 NeuralCF 扩展依赖 PyTorch。项目同时提供 environment.yml 与 requirements.txt 两种依赖声明文件，列明了所有依赖包及版本，便于在 conda 或 pip 环境下复现。

10.2 数据与目录结构

- MovieLens 100K 原始数据位于数据目录，由 src/data_loader.py 加载；
- 本地电影海报位于“电影照片/”目录，由 src/posters.py 匹配；
- 图表输出位于 figures/ 目录，系统截图位于 report/screenshot/ 目录；
- SQLite 数据库文件 app.db 在系统首次运行时自动创建并落库。

10.3 运行步骤

(1) 安装依赖（任选其一）：

```
conda env create -f environment.yml
pip install -r requirements.txt
```

(2) 复现数据分析与模型评估（执行 notebook）：

```
jupyter nbconvert --to notebook --execute
notebooks/01_movielens_recommendation.ipynb
```

(3) 启动 Web 应用：

```
streamlit run app.py
```

启动后在浏览器中访问 FilmTrace 系统，即可进入用户端或管理员端。项目源代码托管于 GitHub 仓库：<https://github.com/xuanlv361-ctrl/MovieLens-Recommendation-System>，包含完整的代码、图表、截图与报告材料。

10.4 可复现性声明

项目中所有涉及随机过程的步骤（训练/测试划分、Random 基线、Top-N 用户抽样等）统一使用 src/config.py 中的 RANDOM_STATE=42；所有模型预测评分均裁剪至 [1, 5]；代码遵循 PEP8 规范，关键函数与类均含 docstring，未使用硬编码路径。

notebooks/01_movielens_recommendation_executed.ipynb 为已执行完成的副本，可作为评估结果的复现依据。

11 团队分工

本项目由 3 名成员协作完成，分工与工作量占比如表 10 所示。

表 10 团队分工与工作量占比

负责人	工作模块	主要内容	工作量占比
吕璇	数据处理、推荐算法、系统开发、模型评估、可视化与报告整理	负责 MovieLens 100K 数据读取与清洗、评分矩阵构建、电影类型处理、海报资源匹配；实现 UserCF、ItemCF、SVD 与混合推荐算法；完成 Streamlit 用户端和管理员端主要功能开发，包括欢迎页、电影库、个性化推荐、评分记录、我的画像、系统统计、模型评估等模块；完成 MAE、RMSE、Precision@K、Recall@K、HitRate@K、NDCG@K 等指标计算与结果分析；整理系统截图、ER 图、系统架构图、Git 提交记录，并参与项目报告撰写。	50%
陈琳	待填写	待填写	待填写
傅梦娇	待填写	待填写	待填写

注：陈琳、傅梦娇的具体分工与工作量占比请根据实际情况补充填写，三人工作量占比之和应为 100%。

12 Git 版本控制与项目管理

本项目全程使用 Git 进行版本控制，并托管于 GitHub。提交历史覆盖了从数据处理、算法实现、系统开发到报告撰写的项目全周期，能够反映项目的迭代开发过程与各阶段的工作演进。图 34 为项目的 Git 提交记录截图。

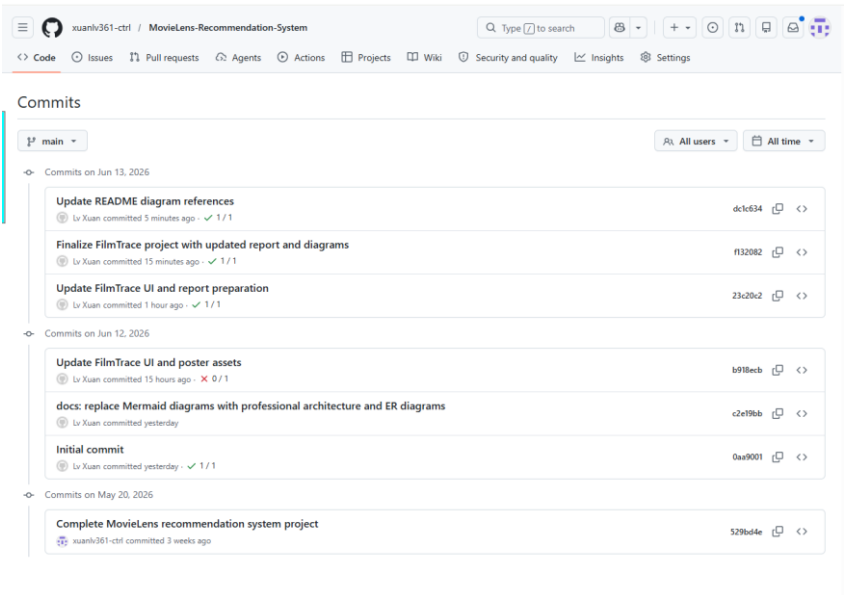


图 34 GitHub 提交记录截图

最终提交的项目仓库包含完整的源代码（数据预处理、模型定义、评估与推理逻辑）、图表、系统截图、ER 图、架构图与报告材料。借助 `git log` 可进一步统计各成员的提交次数与代码改动量，作为团队分工中工作量占比的客观参考依据。

13 结论与展望

13.1 主要工作与发现

本项目基于 MovieLens 100K 数据集，完成了从数据清洗、特征预处理、训练/测试划分，到 9 类推荐模型实现与统一评估的完整流程，并在此基础上构建了基于 Streamlit 与 SQLite 的 FilmTrace 推荐系统，实现了用户端与管理员端的完整功能闭环。主要发现包括：基于偏置残差的 TruncatedSVD 在评分预测上取得最优的 RMSE (0.9826) 与 MAE (0.7763)；在全目录离线 Top-N 协议下，流行度类基线在排序指标上反而领先于个性化模型，这一反直觉结果经分析主要源于评估协议本身与数据高度稀疏，而非个性化算法失效；User-Based CF 在非随机模型中具有最高的目录覆盖率 (0.0916)，体现了准确率与覆盖率之间的权衡。

13.2 系统价值与局限性

在工程价值上，FilmTrace 不止于离线实验，而是落地为一个集账号体系、个性化推荐、评分收藏、用户画像、数据可视化与后台管理于一体的完整系统，具有较好的可演示性与可解释性。其主要局限在于：数据集规模较小且年代较早；离线 Top-N 评估协议可能低估个性化模型的真实排序能力；冷启动问题仅得到缓解；系统尚不具备在线学习与实时反馈能力。

13.3 未来工作

- 改进冷启动推荐：结合更丰富的内容特征与会话信息，提升新用户/新电影的推荐质量；
- 引入深度学习推荐模型：将 NeuralCF 等模型纳入统一评估流程，与传统方法定量对比；
- 改进 Top-N 评估协议：引入采样负例评估或限定候选集，使个性化模型与流行度基线的比较更公平；
- 增强在线能力与实时反馈：引入用户点击、二次评分等实时行为的在线更新机制，并探索在线部署；
- 完善海报与元数据：提升本地海报匹配准确率，丰富电影元数据；
- 扩展至更大数据集：在 MovieLens 1M/10M 等更大规模数据集上验证算法与系统的可扩展性。

参考文献

- [1] Harper F M, Konstan J A. The MovieLens Datasets: History and Context[J]. ACM Transactions on Interactive Intelligent Systems, 2015, 5(4): 1–19.
- [2] Sarwar B, Karypis G, Konstan J, et al. Item-Based Collaborative Filtering Recommendation Algorithms[C]//Proceedings of the 10th International Conference on World Wide Web (WWW). 2001: 285–295.
- [3] Koren Y, Bell R, Volinsky C. Matrix Factorization Techniques for Recommender Systems[J]. Computer, 2009, 42(8): 30–37.
- [4] Herlocker J L, Konstan J A, Terveen L G, et al. Evaluating Collaborative Filtering Recommender Systems[J]. ACM Transactions on Information Systems, 2004, 22(1): 5–53.
- [5] Cremonesi P, Koren Y, Turrin R. Performance of Recommender Algorithms on Top-N Recommendation Tasks[C]//Proceedings of the 4th ACM Conference on Recommender Systems (RecSys). 2010: 39–46.
- [6] Burke R. Hybrid Recommender Systems: Survey and Experiments[J]. User Modeling and User-Adapted Interaction, 2002, 12(4): 331–370.
- [7] He X, Liao L, Zhang H, et al. Neural Collaborative Filtering[C]//Proceedings of the 26th International Conference on World Wide Web (WWW). 2017: 173–182.
- [8] Ricci F, Rokach L, Shapira B. Recommender Systems Handbook[M]. 2nd ed. Boston: Springer, 2015.
- [9] Pedregosa F, Varoquaux G, Gramfort A, et al. Scikit-learn: Machine Learning in Python[J]. Journal of Machine Learning Research, 2011, 12: 2825–2830.
- [10] McKinney W. Data Structures for Statistical Computing in Python[C]//Proceedings of the 9th Python in Science Conference (SciPy). 2010: 56–61.
- [11] Harris C R, Millman K J, van der Walt S J, et al. Array Programming with NumPy[J]. Nature, 2020, 585(7825): 357–362.
- [12] Paszke A, Gross S, Massa F, et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library[C]//Advances in Neural Information Processing Systems (NeurIPS). 2019: 8024–8035.
- [13] Streamlit Inc. Streamlit Documentation[EB/OL]. (2024) [2026-06-14]. <https://docs.streamlit.io>.

附录

附录 A 实验记录与补充图表

除正文呈现的主要评估结果外，项目还记录了以下补充实验，相关图表均保存于 figures/ 目录：SVD 在不同隐因子数 `n_components` 下的 RMSE 变化 (`svd_components_sensitivity.png`)、学习曲线 (`learning_curve.png`)、按评分误差/物品热度/用户活跃度分层的 RMSE 分析 (`error_by_rating_item_cf.png`、`item_tier_rmse.png`、`user_tier_rmse.png`)、按电影类型的 RMSE/MAE 分析 (`genre_rmse.png`) 等，可作为实验记录附录的补充材料。

附录 B 实验记录表（含未达预期的尝试）

下表汇总了项目过程中开展的主要实验，包括关键超参数、结果以及未达预期或被否决的尝试，以满足“记录所有实验（含失败尝试）的超参数与结果”的要求。所有结果均来自执行后的 notebook。

表 11 实验记录表（含未达预期的尝试）

实验	关键超参数	结果	结论 / 是否采用
偏置基线正则化系数搜索	$\text{reg} \in \{0.1, \dots, 10\}$	$\text{reg}=1.0$ 时 RMSE 最低	采用 $\text{reg}=1.0$
UserCF/ItemCF 近邻数搜索	$K \in \{5, 10, \dots, 50\}$	K 过小方差大, K=20 附近趋稳	采用 K=20
SVD 隐因子数搜索	$n_components \in \{10, \dots, 80\}$	n=50 后趋于平稳, 过大易过拟合	采用 n=50
随机评分基线	训练集评分范围均匀采样	RMSE/Top-N 指标最差	仅作质量下界, 未采用
个性化模型用于 Top-N 排序	全目录候选, $\text{rating} \geq 4.0$, K=10	Precision/Recall/NDCG 低于流行度基线	未达预期; 归因于评估协议 (见 9.2)
NeuralCF 深度模型	emb=32, MLP(64, 32), epochs=20, AdamW	RMSE 1.0080, 略逊于 SVD (0.9826)	已实现但未超越 SVD (见 8.6)
全局/随机时间划分	全局时序 / 随机划分	仅作参考, 存在泄漏或不贴近场景	未作为主实验, 仅参考

附录 C 可复现性与随机种子

项目所有随机过程统一使用 `RANDOM_STATE=42`, 预测评分裁剪至 `[1, 5]`, 并提供已执行完成的 notebook 副本 (`notebooks/01_movielens_recommendation_executed.ipynb`) 作为评估结果的复现依据。